
title: Subword Tokenizer & Prototype Training Reference

subtitle: Current status and architecture alignment

date: July 3, 2026

1) Executive Summary

The project currently has two aligned layers:

1. **Tokenizer layer (Rust)**
 - Production BPE tokenizer implementation in ``subword_tokenizer/``.
 - CLI + reusable library API for train/encode/decode/model ops.
2. **Prototype training layer (Python, laptop-first)**
 - ``stream_train.py`` uses the Rust tokenizer model (``model_32k.json``) and trains a small Transformer on
 - Includes checkpointing, step checkpointing, and automated evaluation scripts.

The active path is **Rust tokenizer + Python prototype training**. Legacy Rust/C++ wording is obsolete.

2) What Changed Recently

- Tokenizer usage was enforced in training (removed GPT-2/tiktoken dependency path).
- Added prototype modes suitable for Mac GPU constraints.
- Added loop automation:
 - ``scripts/run_prototype_3x_and_select_best.sh``
 - ``scripts/evaluate_checkpoints.py``
- Added longer-run preset:
 - ``scripts/run_prototype_long_4h.sh``
- Added stream resilience in training:
 - retry/backoff for transient arXiv network timeouts.

3) Current Architecture (Aligned)

Source architecture diagram is maintained in ``doc/architecture.md``.

High-level flow

1. **Data source**
 - Live arXiv stream (primary prototype path), with retry/backoff.
2. **Tokenization**
 - Rust CLI tokenizer (``bpe-tokenizer``) using model ``subword_tokenizer/model_32k.json``.
3. **Model training (prototype)**
 - Tiny Transformer in ``stream_train.py`` (MPS/CUDA/CPU auto device).
4. **Checkpointing & evaluation**
 - Periodic step checkpoints + end checkpoint.
 - Multi-run ranking by eval loss (``best_checkpoint_*.json``).

4) Tokenizer CLI Commands (Current)

From ``subword_tokenizer/``:

- `cargo run --bin bpe-tokenizer -- train <corpus.txt> <vocab\_size>`
- `cargo run --bin bpe-tokenizer -- expand <corpus.txt> <vocab\_size>`
- `cargo run --bin bpe-tokenizer -- tokenize <text>`
- `cargo run --bin bpe-tokenizer -- decode <comma\_separated\_ids>`
- `cargo run --bin bpe-tokenizer -- count`
- `cargo run --bin bpe-tokenizer -- prepare <input.txt> --train <train.bin> --val <val.bin> --test <test.bin>`

Notes:

- Active runtime model file for CLI is `model.json`.
- Training scripts activate the desired model by copying from `model\_32k.json` to `model.json` before tokenization.

5) Prototype Training Commands (Current)

From repository root:

- Quick prototype: `./scripts/run\_prototype.sh`
- 3-run compare and auto-select: `./scripts/run\_prototype\_3x\_and\_select\_best.sh`
- Long run (~3–4h target on M3 Pro): `./scripts/run\_prototype\_long\_4h.sh`
- Evaluate existing checkpoints: `./scripts/run\_evaluation.sh`

6) Validation Status

- Rust tokenizer tests are passing (`src/tests.rs`).
- Prototype training on Mac MPS is validated.
- Best-checkpoint reports are being generated under `checkpoints/`.
- Long-run path now includes network timeout retry/backoff and step checkpoint safety.

7) Current Risks / Observations

1. ****Live-stream dependency risk****: arXiv network timeouts can still pause long runs (now retried).
2. ****Artifact growth****: checkpoints and corpora can grow quickly.
3. ****Documentation drift risk****: ensure markdown-to-PDF sync is part of workflow.

8) Recommended Next Actions

1. Add offline dataset training mode (train from local predownloaded corpus).
2. Add resume-from-checkpoint support for interrupted long runs.
3. Add CI/doc task to regenerate this PDF from markdown automatically.
4. Track large artifacts through retention/LFS policy.

9) File-Level Reference

- `subword\_tokenizer/src/lib.rs`: tokenizer core and model operations
- `subword\_tokenizer/src/main.rs`: CLI dispatch (`bpe-tokenizer`)
- `stream\_train.py`: prototype streaming training loop
- `scripts/evaluate\_checkpoints.py`: checkpoint ranking
- `scripts/run\_prototype\_3x\_and\_select\_best.sh`: train+select workflow

- `scripts/run\_prototype\_long\_4h.sh`: long prototype preset

Prepared on July 3, 2026 for end-of-day status handoff.